Part 1: SQL Joins Practice Problems

1. Create a list of countries and the name of their currency using a natural join

   ```
   SELECT country_name, currency_name
   FROM countries
   NATURAL JOIN currencies;
   ```

2. Create a cross join between the employees and departments tables showing employee names and department names.

   ```
   SELECT e.first_name, e.last_name, d.department_name
   FROM employees e
   CROSS JOIN departments d;
   ```

3. Use the USING clause to create a list of department names and their monthly costs (salaries added up).

   ```
   SELECT department_name, SUM(salary) AS total_monthly_cost
   FROM employees
   JOIN departments USING (department_id)
   GROUP BY department_name;
   ```

4. Create a list of Global Fast Food customer's names and the dates they placed orders and the order total amount.

   ```
   SELECT c.first_name, c.last_name, o.order_date, o.order_total
   FROM f_customers c
   JOIN f_orders o ON c.id = o.cust_id;
   ```

5. Using the DJs on Demand database, create a list of events and the corresponding package code based on that event's cost

   ```
   SELECT e.name AS event_name, e.cost, p.code AS package_code
   FROM d_events e
   JOIN d_packages p ON e.cost BETWEEN p.low_range AND p.high_range;
   ```

6. Create a listing of countries and the spoken language name.

```
SELECT c.country_name, l.language_name
FROM countries c
JOIN spoken_languages l ON c.country_id = l.country_id;
```

7. Using the DJs on Demand database, create a list of all client's names, whether they have booked an event or not, if they have, show the event name and description.

```
SELECT c.first_name, c.last_name, e.name AS event_name, e.description
FROM d_clients c
LEFT OUTER JOIN d_events e ON c.id = e.client_id;
```

8. Create a list of songs titles and the event id whose play list they are being played at, include all songs even if not being used in a play list.

```
SELECT s.title, p.event_id
FROM d_songs s
LEFT OUTER JOIN d_play_list_items p ON s.id = p.song_id;
```

9. Using the previous query, now include the event name as well, continue to display all Songs.

```
SELECT s.title, p.event_id, e.name AS event_name
FROM d_songs s
LEFT OUTER JOIN d_play_list_items p ON s.id = p.song_id
LEFT OUTER JOIN d_events e ON p.event_id = e.id;
```

10. Using the Global Fast Foods database, list the names of the staff members and who their manager is.

```
SELECT s.first_name || '   ' || s.last_name AS staff_member,
m.first_name || ' ' || m.last_name AS manager
FROM f_staffs s
JOIN f_staffs m ON s.manager_id = m.id;
```

11. This time using a hierarchical query, show a list of Global Fast Foods staff members and who their manager is.

```
SELECT first_name || ' ' || last_name AS staff_member,
PRIOR (first_name || ' ' || last_name) AS manager,
LEVEL
FROM f_staffs
START WITH manager_id IS NULL
CONNECT BY PRIOR id = manager_id;
```

12. Using Oracle proprietary syntax, create a list of every employee and their related job title sorted by job_title.

```
SELECT e.first_name, e.last_name, j.job_title
FROM employees e, jobs j
WHERE e.job_id = j.job_id
ORDER BY j.job_title;
```

13. This time, using Oracle syntax, create a list of songs titles and the event id whose play list they are being played at, include all songs even if not being used in a play list.

```
SELECT s.title, p.event_id
FROM d_songs s, d_play_list_items p
WHERE s.id = p.song_id (+);
```

SQL Subquery Practice Problems

1. Create a list of employees first and last names and hire dates but only if they were hired before Bruce Ernst.

```
SELECT first_name, last_name, hire_date
FROM employees
WHERE hire_date < (
        SELECT hire_date
        FROM employees
        WHERE first_name = 'Bruce' AND last_name = 'Ernst'
 );
```

2. Create a list of each employee first initial and last name, salary, and department id for each employee earning more than the average salary of the company.

```
SELECT SUBSTR(first_name, 1, 1) AS first_initial, last_name, salary,
department_id
FROM employees
WHERE salary &gt; (
SELECT AVG(salary)
FROM employees
);
```

3. Now show only those employees whose salary is greater than the highest salary in department 60.

```
SELECT first_name, last_name, salary, department_id
FROM employees
WHERE salary &gt; (
SELECT MAX(salary)
FROM employees
WHERE department_id = 60
);
```

4. Show a list of employees who make the same salary as an employee in department 60.

```
SELECT first_name, last_name, salary, department_id
FROM employees
WHERE salary IN (

SELECT salary
FROM employees
WHERE department_id = 60
);
```

5. Now show the same list but for all employees who make more than any employee in department 60.

```
SELECT first_name, last_name, salary, department_id
FROM employees
WHERE salary &amp;gt; ANY (
SELECT salary
FROM employees
WHERE department_id = 60
);
```

6. Now show the same list but for all employees who make more than all the employees in department 60.

```
SELECT first_name, last_name, salary, department_id
FROM employees
WHERE salary &amp;gt; ALL (
SELECT salary
FROM employees
WHERE department_id = 60
);
```